TP 7 : PROGRAMMATION ORIENTEE OBJET, FLUX ET FICHIERS

Langage C++

Consigne: Le travail est à rendre sur Moodle au plus tard une semaine après la séance.

Dans ce TP, vous implémenter un mini-système de fichiers virtuel en C++ qui permet:

- La gestion hiérarchique des fichiers et dossiers.
- Le stockage persistant du système de fichiers dans un disque virtuel (`system_fs.dat`).
- La navigation entre dossiers (`cd`), la liste des fichiers (`ls`), l'affichage du répertoire courant (`pwd`), la création de dossier (`mkdir`), et la suppression (`rm`).
- L'affichage du contenu des fichiers (`cat`), la création de fichier (`touch`), la modification de fichier (`edit`).
- Chaque opération sur le système de fichier doit être journalisée dans un fichier log

En partant du code fourni, vous devez implémenter toutes les méthodes définies y compris les constructeurs et destructeurs.

1. **Gestion des logs (`Logger`)**
   1.1. Complétez la méthode `log()` pour écrire les actions effectuées dans un fichier.
   1.2. Complétez la fonction `fermer()`
2. **Gestion des fichiers (`Fichier`)**
   2.1. Implémentez `ecrire()` pour permettre l'ajout de texte à un fichier.
   2.2. Implémentez `afficher()` pour afficher le contenu du fichier.
   2.3. Implémentez la méthode amie qui surcharge l'opérateur >> permettant de saisir les informations d'un fichier
3. **Gestion des dossiers (`Dossier`)**
   3.1. Implémentez les fonctions d'ajout de fichier et de dossier
   3.2. Implémentez les fonctions permetttant de supprimer un fichier et un dossier
   3.3. Implémentez la fonction `afficher()` qui liste le contenu d'un dossier
   3.4. Implémentez la méthode amie qui surcharge l'opérateur >> permettant de saisir les informations d'un dossier
4. **Disque Virtuel (`DisqueVirtuel`)**
   4.1. Implémentez `sauvegarder()` et `charger()` pour stocker et récupérer le système de fichiers en binaire (`systeme_fs.dat`). L'ensemble des fichiers, dossiers ainsi que la hiérarchie des dossiers et fichiers doivent être sauvegardé dans ce fichier. Le programme devra également pouvoir restaurer cet état en chargeant le fichier binaire.

4.2. Implémentez la fonction `serialiserDossier()` qui prend en entrée un flux fichier ouvert et un dossier à sauvegarder et sérialise l'ensemble du dossier ainsi que les sous dossiers et fichiers contenus. Cette fonction est appelée par sauvegarder(), elle doit écrire dans le fichier l'état exacte du système. Vous proposez une approche pour représenter puis écrire l'état du système.
Privilégiez l'utilisation d'un fichier binaire, ou d'un fichier JSON (https://github.com/nlohmann/json) pour représenter l'état du système.

4.3. Implémentez la fonction `deserialiserDossier()` qui prend en entrée un flux fichier et un dossier vide et restore dans ce dossier vide l'état du système lu à partir du flux fichier. Cette fonction permet le décodage de la représentation de l'état du système obtenue avec la fonction `serialiserDossier()`. Il est donc possible de sauvegarder l'état du système de fichier après fermeture du programme et de le restaurer également lorsqu'on lance à nouveau le programme.

5. Gestion du système de fichier (`SystemeFichier`)

5.1. Implémentez toutes les méthodes de la classe permettant ainsi de doter notre système de fichier de commandes basiques.

6. Interface utilisateur (Shell)

6.1. Implémentez la fonction main() pour gérer les commandes utilisateur:

- ls -> afficher les fichiers et dossiers du dossier courant
- cd [dossier] -> changer de dossier
- mkdir [nom] -> créer un dossier
- touch [nom] -> créer et éditer un fichier
- edit [nom] -> éditer un fichier
- rm [nom] -> supprimer un fichier ou un dossier
- cat [nom] -> afficher le contenu d'un fichier
- pwd -> afficher le chemin jusqu'au dossier courant
- exit -> quitter

6.2. À la sortie du programme vous devez appeler la fonction sauvegarder du disque virtuel afin de sauvegarder l'état actuel du système.

6.3. Au démarrage du programme vous vérifiez si le fichier du dsique virtuel existe, si oui, vous chargez l'état du système.